SOFTWARE REQUIREMENTS SPECIFICATION

# Money Manager Application

A Professional Python Desktop Finance Application

Built on 3-Tier Architecture + SOLID Principles

Python 3.10+ | SQLite | PySide6 | v1.0

Software Architecture Team · Version 1.0 · March 2024

# Project Scope

## Desktop Application

Built using Python for Windows, Mac, and Linux platforms

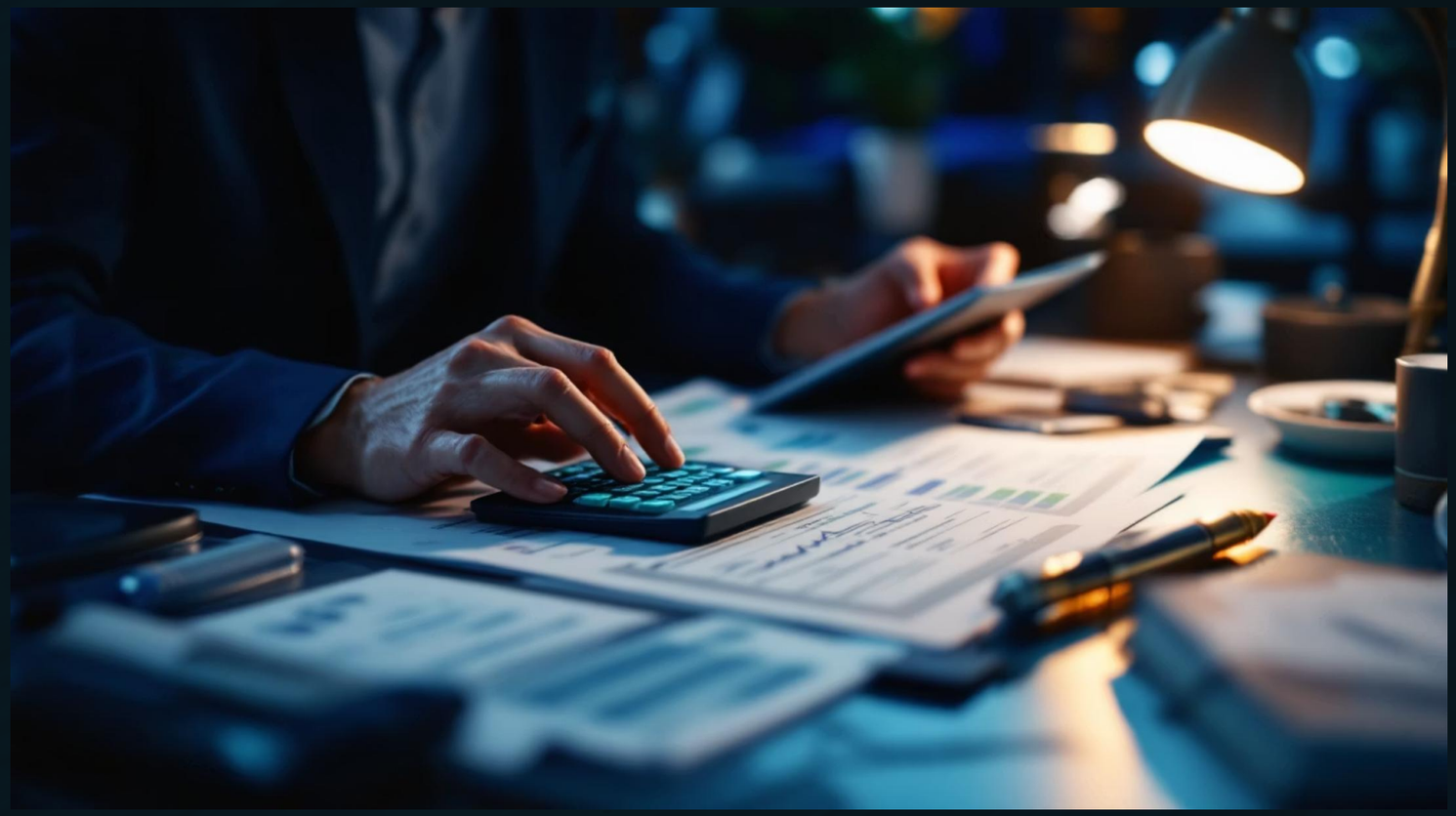
## Offline-First

Full functionality without internet connection for privacy and reliability

## Data Management

Secure local storage with backup capabilities

# Project Overview



## Project Purpose

A personal finance management desktop application designed to help users track their financial health with precision and ease.

## Core Capabilities

- Track income and expenses in real-time
- Organize and manage monthly budgets
- Generate comprehensive financial reports
- Visualize spending patterns with charts

TECH STACK

# Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| **GUI** | PySide6 | *Cross-platform desktop UI* |
| **Services** | Python 3.10+ | *Business logic & validation* |
| **Database** | SQLite 3 | *Embedded relational database* |
| **Build / Dist** | PyInstaller | *Self-contained executable build* |

## INTRODUCTION

| | |
|---|---|
| **Project Name** | Money Manager App |
| **Language** | Python 3.10+ |
| **Architecture** | 3-Tier + SOLID Principles |
| **Database** | SQLite (local-first) |
| **GUI Framework** | PySide6 |
| **Future Plans** | Android · Web · Cloud Sync |

AGENDA

# What We'll Cover


01 Introduction & Objectives

02 System Architecture

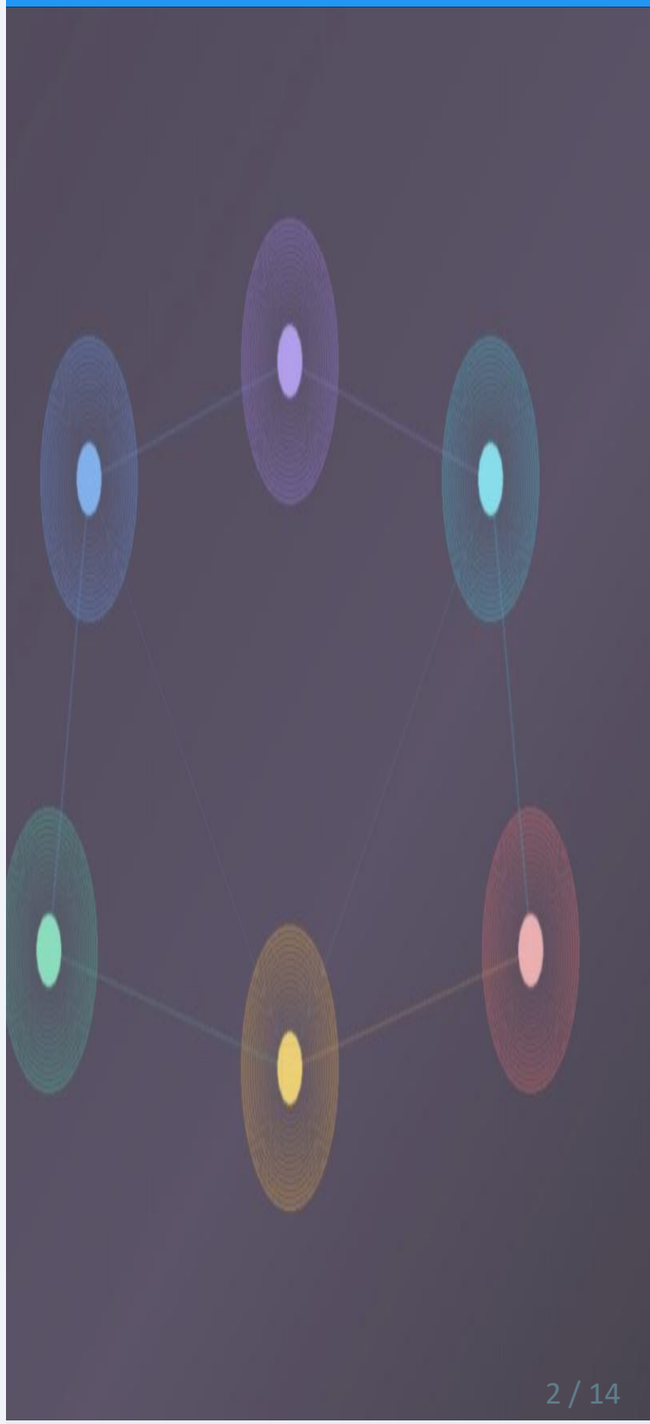
03 SOLID Principles

04 Core Features

05 Database Design

06 Requirements

07 Future Roadmap

OBJECTIVES

# Project Objectives

### Track Finances

Record expenses with full categorization

### Visual Reports

Dashboard with charts and spending trends

### Budget Control

Per-category monthly limits with alerts

### Goals

Personal reminders and savings goal tracking

### Data Integrity

Reliable SQLite storage with ACID compliance

### Future-Ready

Clean architecture for Android & Web expansion

## ARCHITECTURE

### PRESENTATION LAYER

*«GUI Layer»*

*PySide6*

▼ *calls / delegates*

### BUSINESS LOGIC LAYER

*«Service Layer»*

*TransactionService · BudgetService · UserService*

▼ *calls / delegates*

### DATA LAYER

*«Repository Layer»*

*user_repo · TransactionRepo · BudgetRepo*

# SOLID Design Principles

## S

**Single Responsibility**

Each class has one job only

TransactionService
BudgetRepo

## O

**Open / Closed**

Extend features via new classes

not by modifying existing ones

## L

**Liskov Substitution**

SQLiteRepo can swap to APIRepo

without breaking services

## I

**Interface Segregation**

Small focused interfaces only

IBudgetRepo
ITransactionRepo

## D

**Dependency Inversion**

Services depend on abstractions

not concrete implementations

# Database Design

Relational database schema with user-centric data organization ensuring data integrity and efficient queries.

1

### Users Table

User profiles storing credentials, preferences, and personal information

2

### Transactions Table

Financial operations including amounts, categories, dates, and descriptions

3

### Budgets Table

Monthly spending limits by category with tracking periods

4

### Notes Table

Additional transaction details, comments, and documentation

5

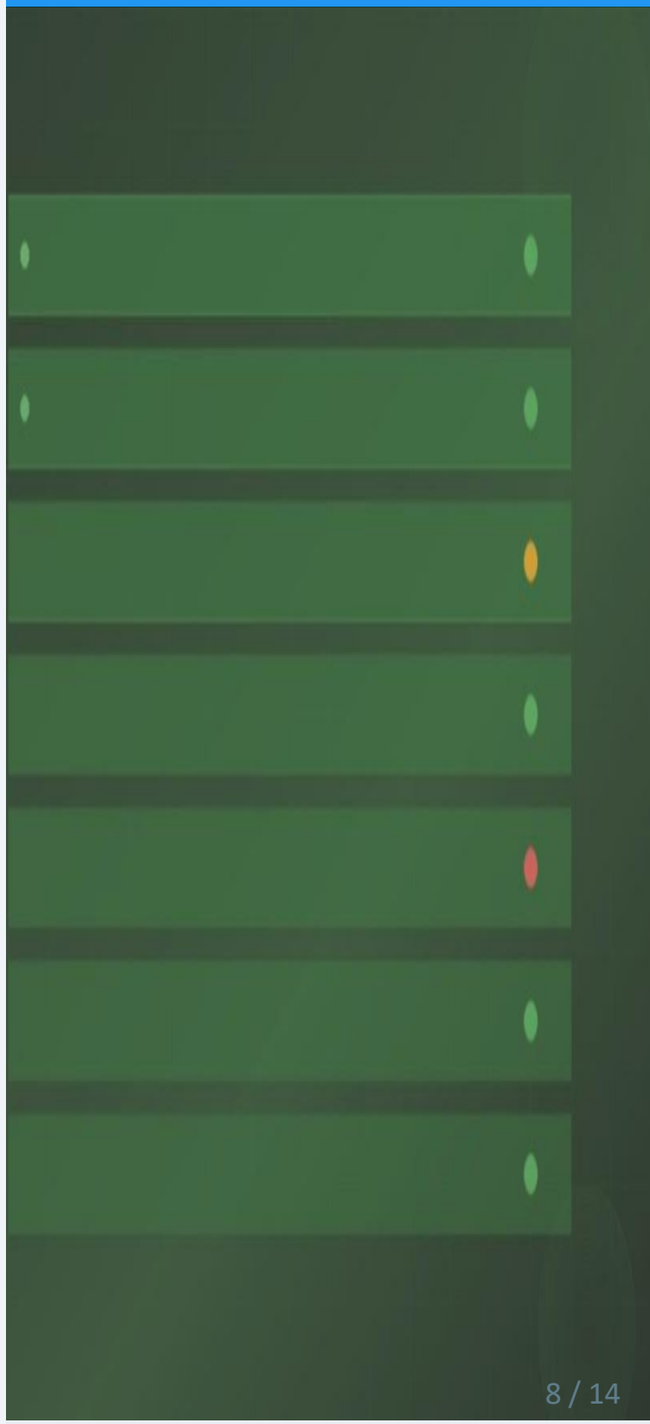
### Goals Table

Savings targets with amounts, deadlines, and progress tracking

DATABASE DESIGN

# Database Schema

| | |
|---|---|
| **users** | *id, username, password* |
| **transactions** | *id, amount, category, Month, Year,  total_amount* |
| **budgets** | *id, amount, Month, Year* |



## Users

PK User_ID

Username

Password

## Budget

PK Budget_ID

Name

Amount

Month

Year

## Transactions

PK Transaction_ID

Name

Amount

Category

Month

Year

total_amount

NFR

# Non-Functional Requirements

### Performance

< 3s launch · < 200ms UI

### Reliability

Atomic writes · no data loss

### Usability

Core tasks in < 5 min, no docs

### Security

bcrypt hashing · no plaintext

### Portability

Windows · macOS · Ubuntu 20.04+

### Scalability

SQLite → REST API with 0 changes

### Maintainability

> 80% unit test coverage

### Data Integrity

FK constraints · cascade deletes

USE CASES

**UC-01**

## Add Transaction

Actor → GUI → Service → Repository → DB
Validates input, persists, refreshes dashboard

**UC-02**

## Monitor Budget

System alerts at 80% and 100% of limit
Real-time spending vs budget comparison

**UC-03**

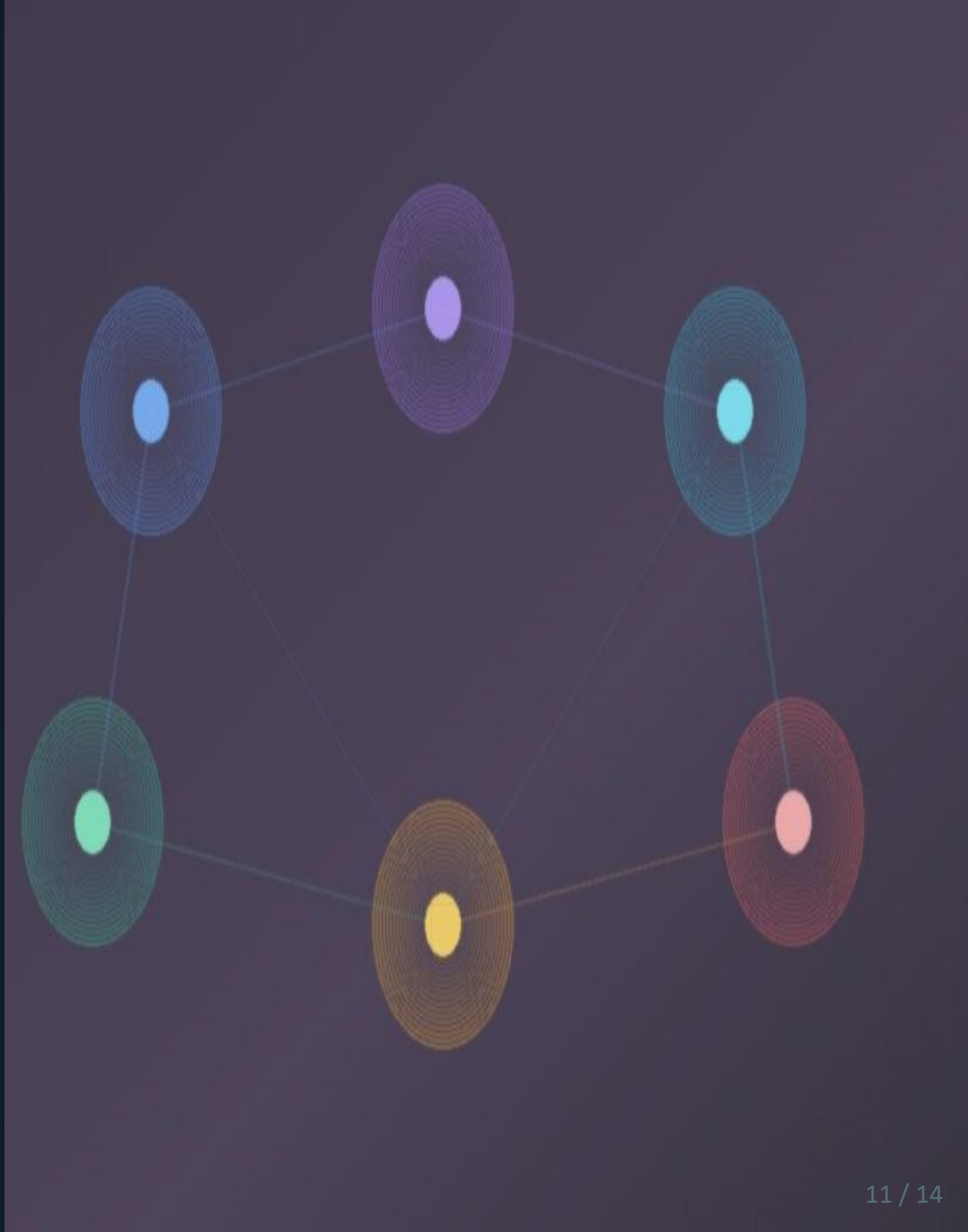
## View Dashboard

Aggregates transactions, renders charts
KPI cards with monthly trends

**UC-04**

## Set Savings Goal

Define target amount + deadline
System tracks progress and sends reminders



# Key Features

### Saving & Expense Tracking

Log transactions with categories, dates, and descriptions. Track cash and digital payments seamlessly.

### Monthly Budget Management

Set spending limits by category. Get alerts when approaching or exceeding budget thresholds.

### Interactive Dashboard

Real-time overview of financial health with summary cards and quick access to key metrics.

### Export Reports

Generate detailed financial reports in PDF and Excel formats for record keeping and analysis.

### Multi-Currency Support

Manage finances in different currencies with automatic conversion rates and currency-specific tracking.

### Financial Goals

Set savings targets and track progress toward specific financial objectives with milestone tracking.

# Users & Stakeholders

1

## End User

Individual managing personal finances, tracking daily expenses, and planning budgets

2

## Developer

Python engineer building and maintaining application codebase and features

3

## Quality Assurance

Tester ensuring functionality meets requirements and identifying bugs before release

4

## Product Owner

Project manager defining requirements, prioritizing features, and approving deliverables

# System Architecture

A three-tier architecture ensuring separation of concerns and maintainable code structure.

## Presentation Layer

User interface built with modern GUI framework. Handles user input, displays data, and manages screen navigation.

## Business Logic Layer

Core application logic processing transactions, validating rules, calculating budgets, and generating reports.

## Data Layer

Database management system storing all financial data securely with efficient queries and backup mechanisms.

REQUIREMENTS

# Functional Requirements

## FR-01 · Transactions

- Add / edit / delete transactions
- Filter by date, category, type
- Auto-categorization by keyword

## FR-02 · Budgets

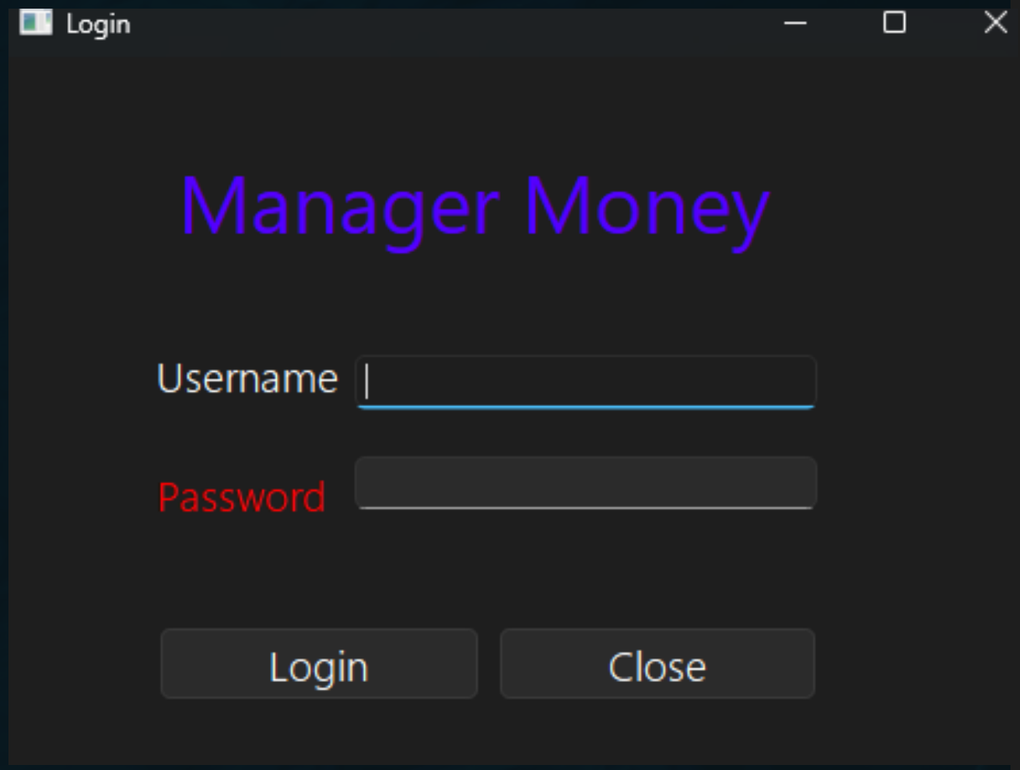
- Per-category monthly budgets
- 80% and 100% over-limit alerts
- Real-time spending progress

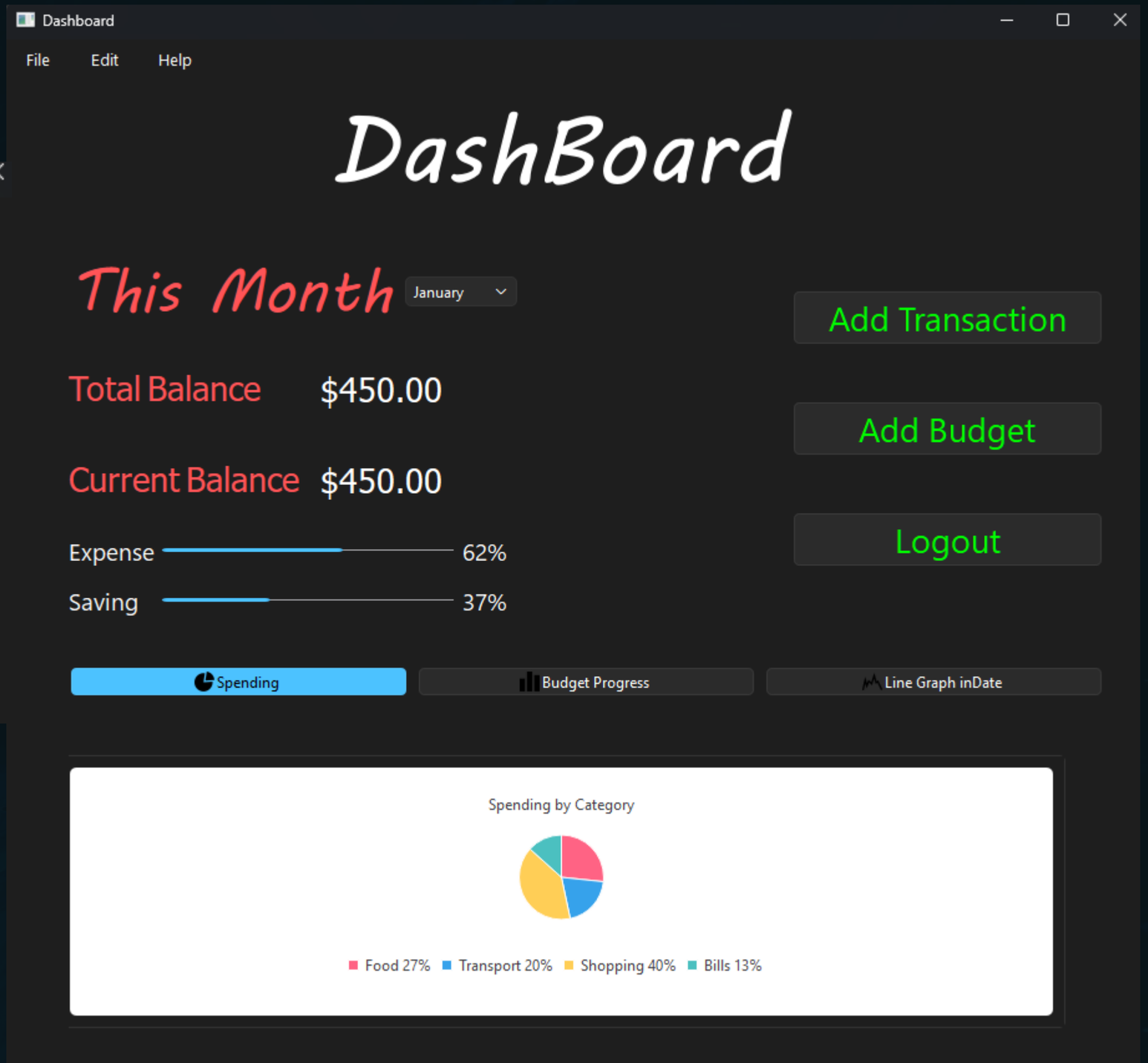
## FR-03 · Dashboard

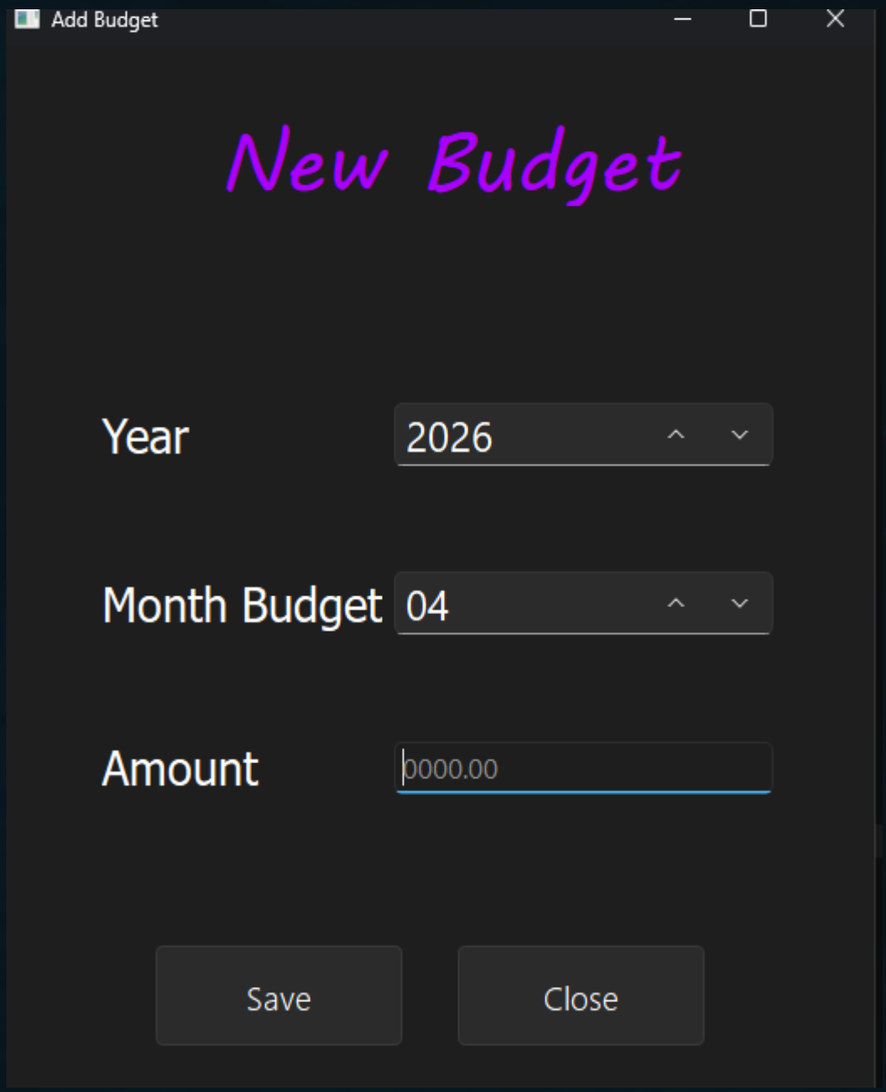
- Monthly KPI summary cards
- Pie & bar chart visualizations
- PDF / Excel report export

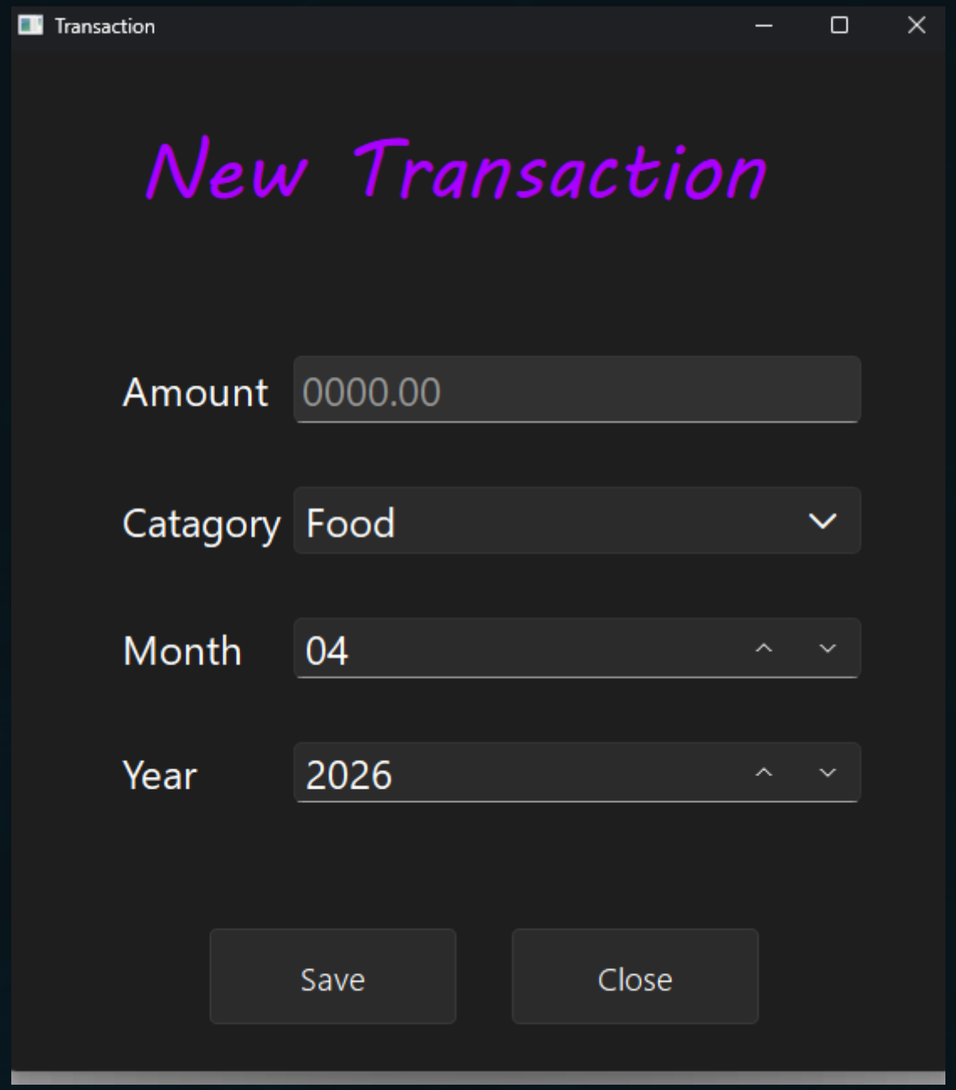
## FR-04 · Goals

- Tagged notes with reminders
- Savings goals with % progress
- Deadline tracking & alerts


Login
Manager Money
Username
Password
Login
Close


Dashboard
File
Edit
Help
DashBoard
This Month
January
Add Transaction
Total Balance
$450.00
Add Budget
Current Balance
$450.00
Logout
Expense
62%
Saving
37%
Spending
Budget Progress
Line Graph inDate
Spending by Category
Food 27%
Transport 20%
Shopping 40%
Bills 13%


Add Budget
New Budget
Year
2026
Month Budget
04
Amount
0000.00
Save
Close


Transaction
New Transaction
Amount
0000.00
Catagory
Food
Month
04
Year
2026
Save
Close

# Future Enhancements

### Mobile App

iOS and Android applications for on-the-go expense tracking

### Web Version

Browser-based access with responsive design for any device

### Cloud Sync

Automatic data synchronization across all devices and platforms

### AI Analysis

Machine learning to predict spending patterns and optimize budgets

ROADMAP

# Future Roadmap

### Current

- ✓ Python Desktop App
- ✓ SQLite Local Storage
- ✓ PyQt5 / PySide6 GUI

### Q3 2026

- ✓ Android App (Kivy)
- ✓ Cloud Sync (Firebase)
- ✓ Multi-user Support

### Q1 2027

- ✓ Web App (FastAPI + React)
- ✓ PDF / Excel Reports
- ✓ AI Spending Insights

# Designed by

| الاسم | الرقم الجامعي |
|---|---|
| زيد عمرو شاهين (شعبة 4) | 202403163 |
| حمزة أسامة خليل ابراهيم (شعبة 2) | 202403038 |
| أنس عماد صبري الشويكي (شعبة 4) | 202500768 |
| محمد احمد محمود حمد (شعبة 4) | 202400324 |
| رويد خالد عبد لافي (شعبة 4) | 202401262 |